

HireFlow — High-Level Design

Architecture, data flow, AI components, RBAC, schema, and deployment for an AI-native, multi-tenant HRMS.

1. System Overview

HireFlow is a multi-tenant HRMS built as a Next.js monolith deployed on Vercel. The App Router serves the React UI and the REST API from the same codebase. **PostgreSQL (Supabase)** is the only persistent store, accessed via Prisma with the native pg adapter. **Anthropic Claude Sonnet 4.6** is invoked directly from API routes — no separate AI service. Every Claude call is wrapped by an `observedAiCall()` telemetry layer that emits per-call tokens, latency, and cost into `AiCallLog`, powering the `SUPER_ADMIN` AgentOps console.

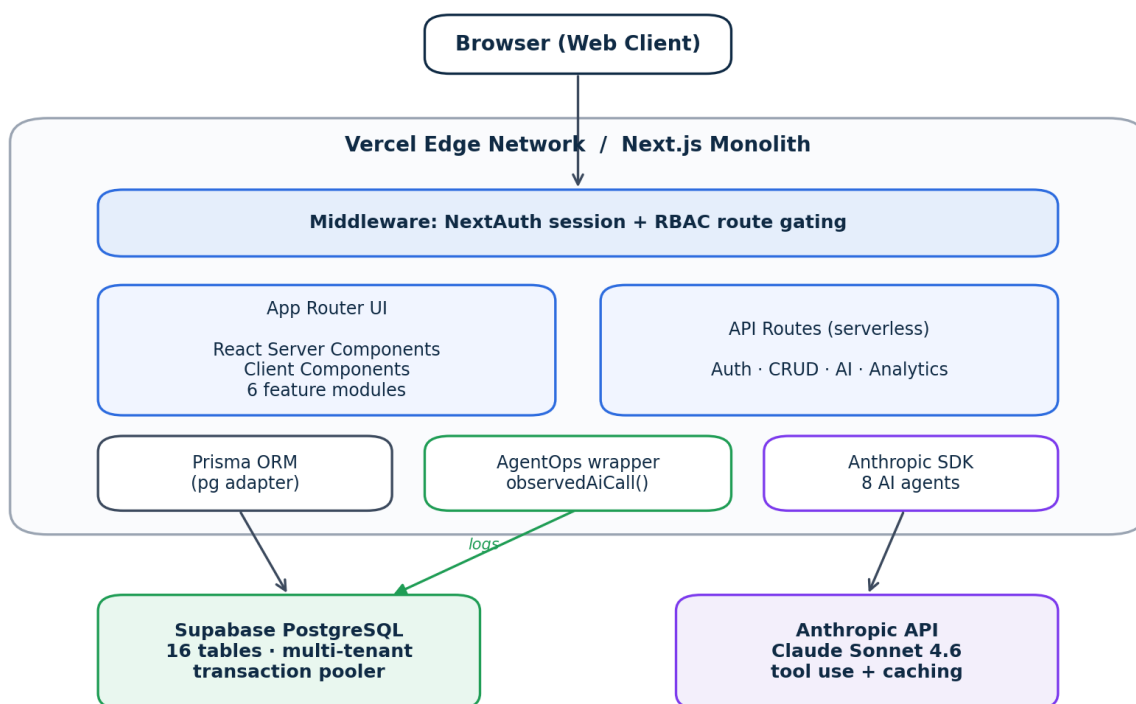


Figure 1. System components — single Next.js monolith on Vercel; two outbound integrations (Postgres + Anthropic).

2. Request Lifecycle

Every request — page or API — passes through middleware-level auth, then an API handler that performs explicit role checks before any data access. AI-bearing routes additionally route through the AgentOps telemetry wrapper.

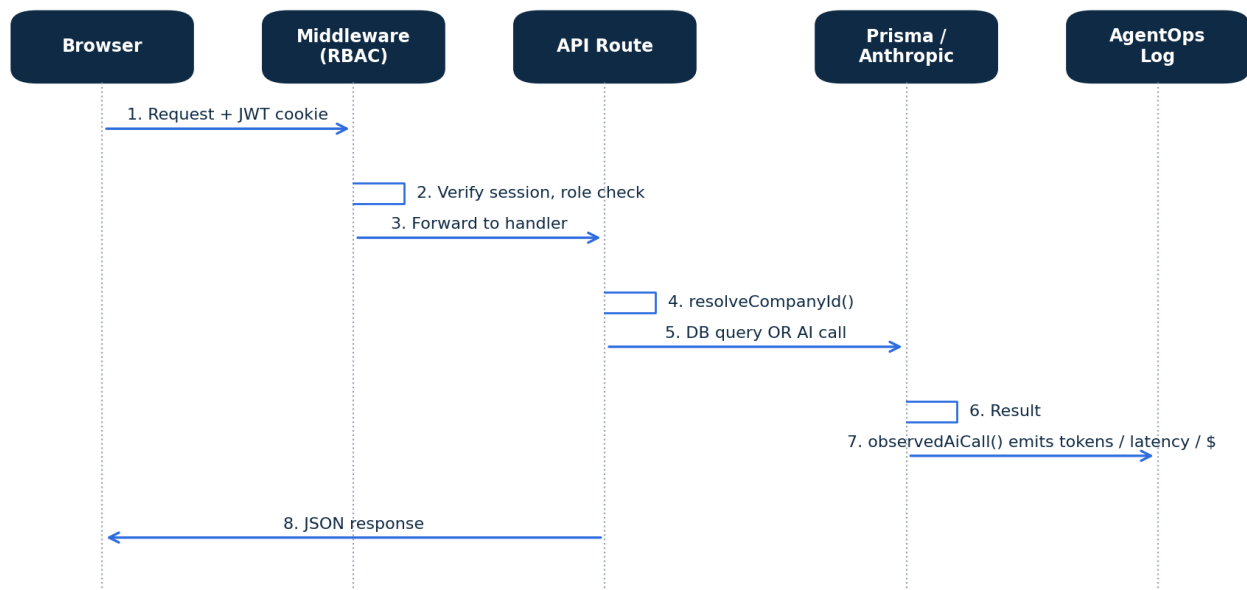


Figure 2. Request lifecycle — RBAC is enforced at four points; AgentOps logging is fire-and-forget so user-facing latency is unaffected by log writes.

3. User Roles & Access Control

Four roles with escalating permissions: **SUPER_ADMIN** (platform / cross-tenant) > **HR_ADMIN** (full HR within company) > **MANAGER** (team-scoped reads & approvals) > **EMPLOYEE** (self-serve). Enforcement is layered four times so a single missed check at one layer cannot leak data:

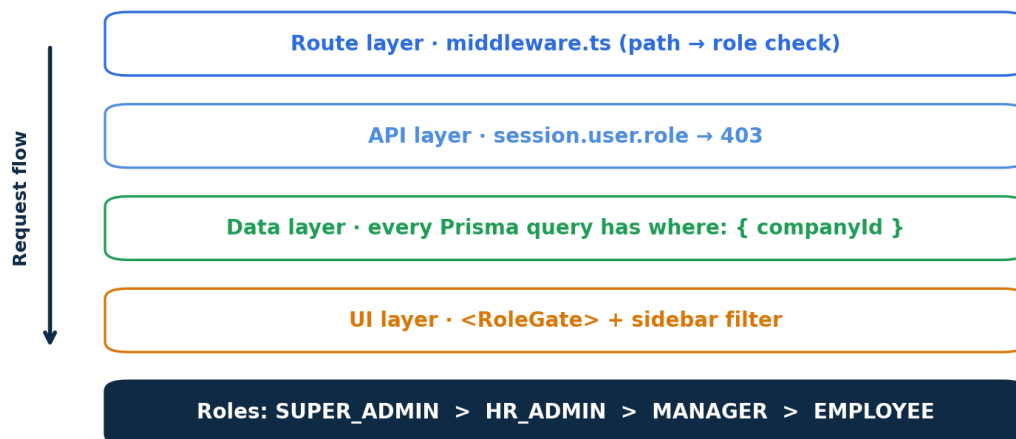


Figure 3. Defense-in-depth RBAC — middleware, handler, query, and UI all carry independent enforcement.

Permission matrix (excerpt from `lib/rbac.ts`)

Permission	SUPER_ADMIN	HR_ADMIN	MANAGER	EMPLOYEE
employees:read	✓	✓	✓ (team)	—
employees:write	✓	✓	—	—
recruitment:read	✓	✓	✓	—
recruitment:write	✓	✓	—	—

Permission	SUPER_ADMIN	HR_ADMIN	MANAGER	EMPLOYEE
leave:read	✓	✓	✓ (team)	✓ (own)
leave:write	✓	✓	✓ (approve)	✓ (request)
tickets:read	✓	✓	—	✓ (own)
analytics:read	✓	✓	—	—
agentops:read	✓	—	—	—

4. Data Flow — Three Critical Paths

4.1 Recruitment pipeline (AI-heavy)

Job creation triggers the JD Parser and Bias Detector in parallel. Candidate uploads stream through the Resume Screener, which produces a numeric score plus structured strengths and gaps. The Kanban board sorts by score; on the candidate detail page, the Interview Generator turns identified gaps into 3–5 targeted questions.

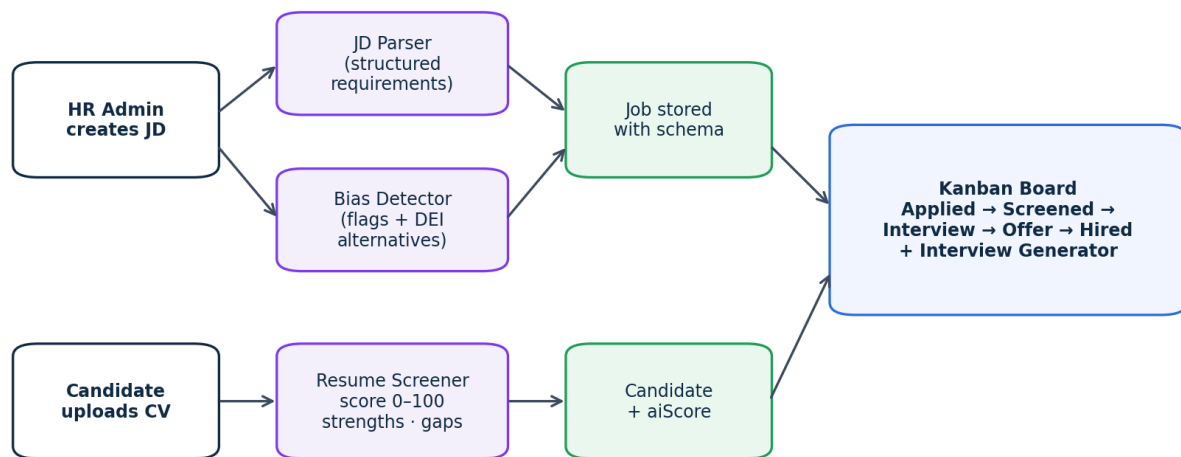


Figure 4. Recruitment pipeline data flow.

4.2 PolicyBot RAG

PolicyBot answers employee policy questions via in-process keyword retrieval (no vector DB). The top three policy chunks are wrapped in an Anthropic ephemeral cache block (5-minute TTL) so repeated questions cost ~10% of an uncached call. The model returns a structured object with sources, confidence, and a `shouldEscalate` flag — when set, an HR ticket is auto-created in the same request.

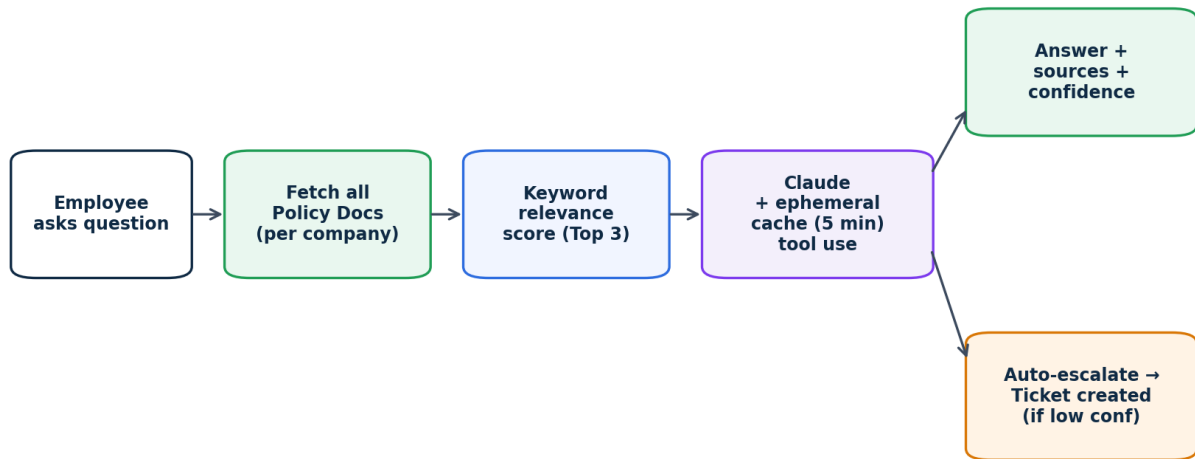


Figure 5. PolicyBot RAG flow with auto-escalation.

4.3 Performance review synthesis

When HR triggers synthesis for an employee in a review cycle, all PerformanceReview records (self / peer / manager) are formatted as structured text and passed to the Review Synthesizer agent, which returns *strengths*, *growth areas*, and a *narrative*. The synthesised JSON is stored on the employee's self-review record and rendered alongside the raw multi-rater inputs.

5. AI Components

Eight agents live in `lib/ai/`. Six use the Anthropic *tool use* API with a single forced tool, which guarantees schema-valid JSON output and removes the need for any regex parsing of model prose. Two simpler agents (Survey Insight, Analytics Narrator) use plain JSON or text prompts.

Agent	File	Trigger	Pattern
jd-parser	lib/ai/recruitment.ts	Job post create	Tool use
bias-detector	lib/ai/recruitment.ts	JD save / on-demand	Tool use
resume-screener	lib/ai/recruitment.ts	Candidate upload	Tool use
interview-generator	lib/ai/recruitment.ts	Candidate detail	Streaming text
policybot	lib/ai/policybot.ts	Employee question	RAG + tool use + cache
review-synthesizer	lib/ai/reviews.ts	HR triggers synthesis	JSON prompt
survey-insight	lib/ai/surveys.ts	Survey results page	JSON prompt
analytics-narrator	lib/ai/analytics.ts	Analytics page load	Text prompt

Tool-use pattern (recruitment example)

```

anthropic.messages.create({
  model: 'claude-sonnet-4-6',
  tools: [{ name: 'score_resume', input_schema: { score, explanation, strengths, gaps } }],
  tool_choice: { type: 'tool', name: 'score_resume' },
  messages: [...]
}) → response.content[0].input // type-safe structured result
  
```

Observability wrapper

```

const result = await observedAiCall({
  agentName: 'resume-screener',
  ...
})
  
```

```

companyId,
fn: () => anthropic.messages.create({ ... }),
})
// captures input/output tokens, latency (ms), cost (USD)
// fire-and-forget write to AiCallLog – never blocks the response

```

Cost basis: Claude Sonnet 4.6 at \$3 per M input tokens, \$15 per M output tokens. Ephemeral prompt caching is applied to stable system prompts and policy context blocks for ~90% cost reduction on repeated calls within a 5-minute cache window.

6. Database Schema (16 models, 3 logical groups)

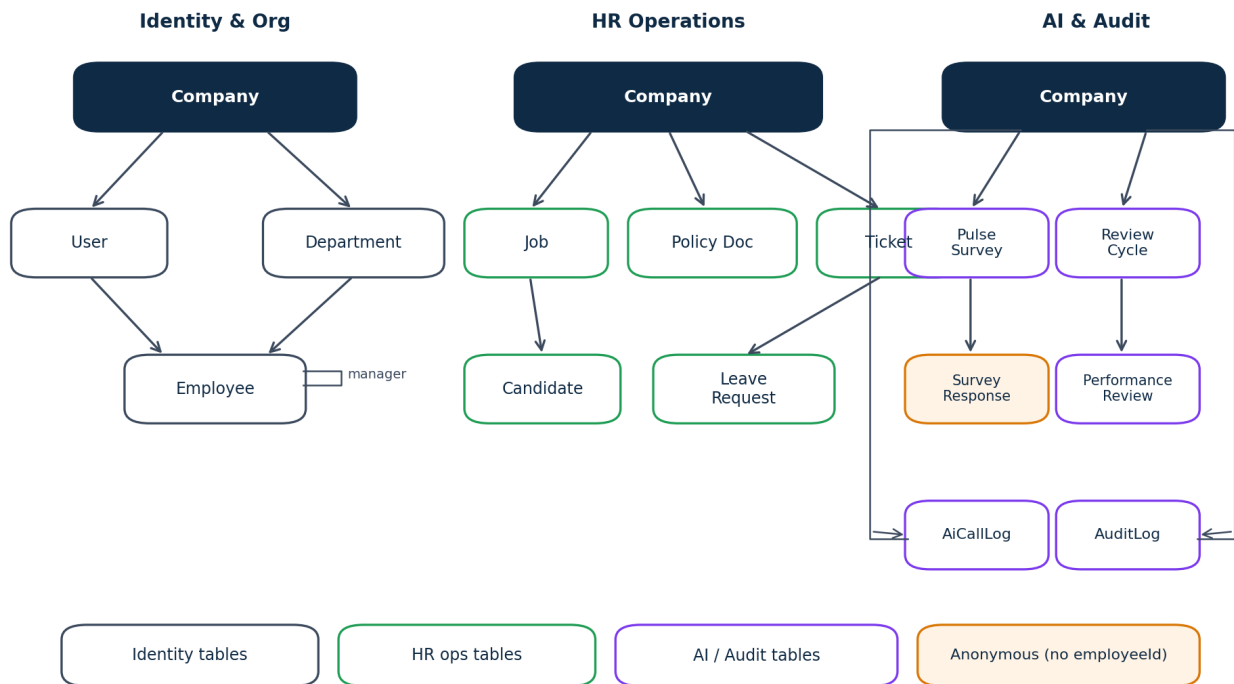


Figure 6. Logical ER view — every business table carries companyId; PulseSurveyResponse deliberately has no employeeId (privacy by schema design).

- All IDs are **CUIDs** (no auto-increment integers — record counts are not leaked).
- **Soft deletes only**: status fields preserve full history for compliance forensics.
- **JSON-stringified arrays** for fields like skills / requirements — parsed in app code; tradeoff for Prisma simplicity.
- **AiCallLog.companyId is nullable** — some calls have no tenant context (platform-level operations).

7. External Integrations

Integration	Status	Notes
Anthropic API	Implemented	Core AI engine — Claude Sonnet 4.6
Supabase PostgreSQL	Implemented	Managed DB + transaction pooler for serverless
Vercel	Implemented	Serverless deployment, Edge CDN, env-var management
NextAuth 5	Implemented	JWT sessions, email-only credentials for demo
SAML / SSO (Okta, Azure AD)	Roadmap	Required before enterprise sale

Integration	Status	Notes
ATS / HRIS (Workday, BambooHR)	Roadmap	Import/export connectors
Payroll (ADP, Gusto)	Roadmap	Bank-grade compliance scope
Email (SendGrid)	Roadmap	Approval / review notifications

8. Deployment Architecture

Code lives on GitHub main; Vercel CI runs `prisma generate` then `next build` and produces stateless Node 20 serverless functions — one per API route. The `pg.Pool` instance is reused within a warm Lambda; cold starts go through Supabase's transaction pooler on port 6543 to avoid connection exhaustion.

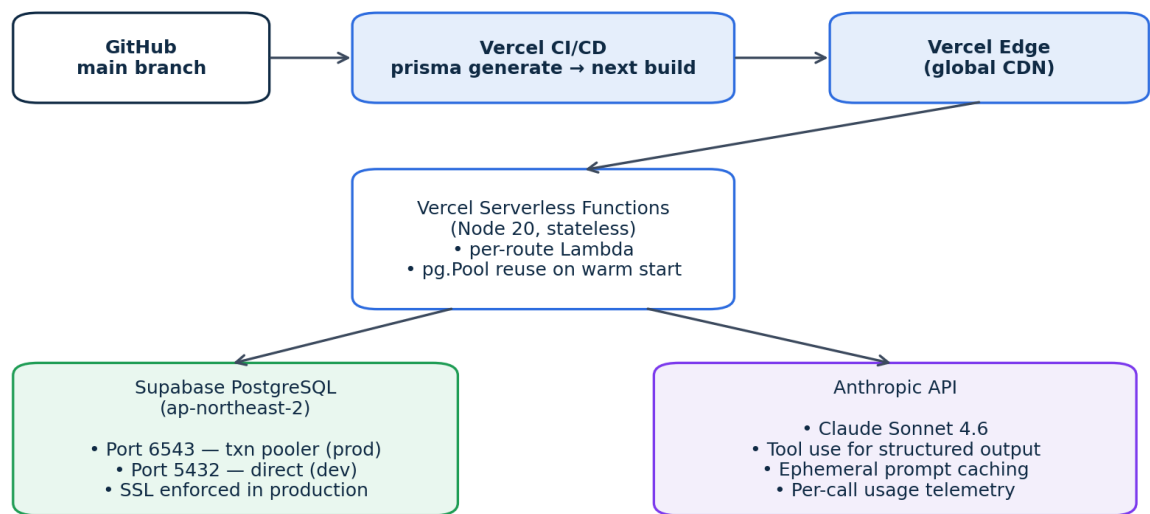


Figure 7. Deployment pipeline and runtime topology.

Required environment variables (Vercel encrypted store)

Variable	Purpose
DATABASE_URL	Supabase transaction pooler connection string
ANTHROPIC_API_KEY	Claude API access
AUTH_SECRET	NextAuth JWT signing key (32+ char random)
NEXTAUTH_URL	Canonical production domain
AUTH_TRUST_HOST	Required for NextAuth on Vercel

9. Non-Functional Posture & Compliance Roadmap

Multi-tenancy: enforced by `companyId` on every table and every Prisma query — cross-tenant access is architecturally impossible. **Audit trail:** the `AuditLog` table records every user action with entity, action type, and diff, enabling forensic review and regulatory export. **Transport security:** TLS end-to-end (Vercel HTTPS + Supabase SSL). **Secrets:** all keys live in Vercel encrypted env vars — zero secrets in the repository. **Observability:** per-call AI cost / latency / token telemetry is queryable in real time by `SUPER_ADMIN`, grouped by agent and by company. **Compliance roadmap (not built):** SOC 2 Type II, GDPR right-to-erasure flows, SSO / SAML via Okta or Azure AD, and regional data residency via an on-prem-hostable model when Anthropic Enterprise tier is not contracted (isolated swap in `lib/ai/client.ts`).